

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1002	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	RAVURU PRANATHI	Reg. No.:	192525076
Date:	15-08-2026	Duration:	60 minutes

Code of Conduct

I R.Pranathi(192525076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *R. Pranathi*

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.
- 6. Commit History Analysis**
 - Present the commit history of the project.

- Explain how commit messages follow good version control practices and contribute to project maintenance.
- 7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
- 8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
- 9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
- 10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
- 11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
- 12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.

Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Analysis

1.1 Problem Statement

The Smart Construction Project Management System is a web-based application designed to help construction companies efficiently manage their construction projects, tasks, workers, materials, deadlines, and project progress.

Traditional construction project management often depends on spreadsheets, paper-based records, phone calls, and manual reporting. These approaches can result in data duplication, delayed updates, difficulty tracking project progress, and poor coordination between project managers and workers.

The proposed system provides a centralized platform where project managers can create and monitor construction projects, assign tasks, manage resources, and track project status.

1.2 Project Objectives

- Provide centralized management of construction projects.
- Allow managers to create and update construction projects.
- Manage construction tasks and assign them to workers.
- Track project progress and deadlines.
- Maintain project-related information in a database.
- Provide a simple and user-friendly web interface.
- Use Git for version control and collaborative development.
- Use Docker for portable and consistent application deployment.

1.3 Functional Requirements

- User / project manager login.
- Project creation.
- Project modification and deletion.
- Project status tracking.
- Task creation and assignment.
- Worker management.
- Material / resource tracking.
- Project deadline management.
- Database storage.
- Project progress monitoring.

1.4 Non-Functional Requirements

- Reliability
- Security
- Maintainability
- Scalability
- Portability
- Performance
- Easy deployment

1.5 Expected Outcome

The expected outcome is a containerized web application that allows construction project managers to manage construction activities efficiently. Git provides controlled version management, while Docker provides a consistent execution environment.

2. Project Structure Design

The following repository structure is proposed:

```
C:\Users\prana>cd C:\Users\prana\smart-construction-project

C:\Users\prana\smart-construction-project>dir
Volume in drive C is Windows
Volume Serial Number is 2876-3C4F

Directory of C:\Users\prana\smart-construction-project

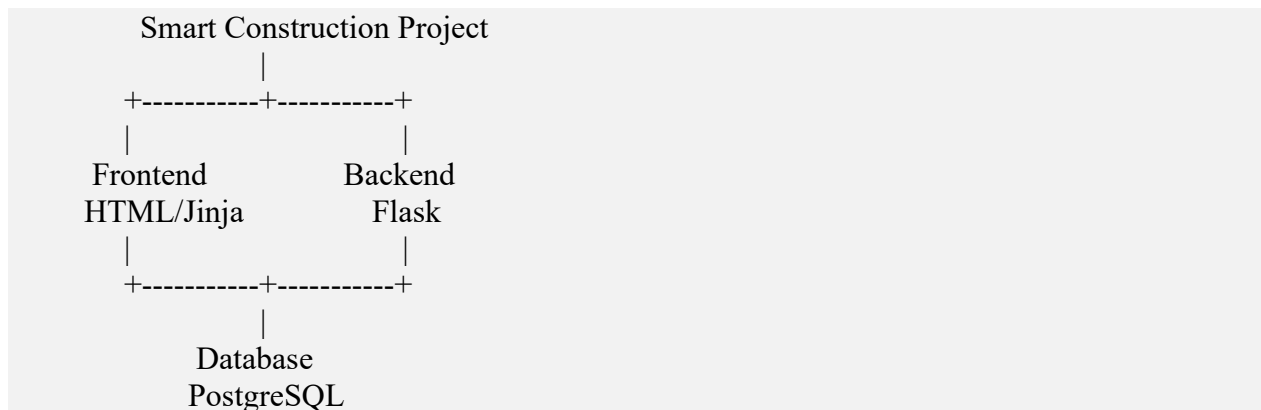
16-08-2026  21:42    <DIR>          .
16-08-2026  21:37    <DIR>          ..
16-08-2026  21:42             49 .dockerignore
15-08-2026  23:42             40 .gitignore
15-08-2026  23:32    <DIR>          app
16-08-2026  21:42          182 Dockerfile
16-08-2026  21:42             37 requirements.txt
15-08-2026  22:54    <DIR>          tests
                  4 File(s)          308 bytes
                  4 Dir(s)  859,281,702,912 bytes free
```

2.1 Folder and File Description

Folder / File	Purpose
app/	Contains application source code
templates/	Contains HTML user-interface files
tests/	Contains application test cases
__init__.py	Initializes the Flask application
routes.py	Defines application routes
models.py	Defines database models
index.html	Main application interface
requirements.txt	Contains Python dependencies
Dockerfile	Defines the application container
docker-compose.yml	Defines application and database services
.dockerignore	Prevents unnecessary files from entering Docker image

.gitignore	Prevents unwanted files from being tracked by Git
README.md	Contains project documentation
.env	Stores environment configuration

2.2 Repository Architecture



3. Git Repository Initialization

Git is used to track changes made to the project and maintain different versions of the source code.

3.1 Git Repository and File Tracking

```

C:\Users\prana\smart-construction-project>git add .

C:\Users\prana\smart-construction-project>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   app/models.py
    new file:   docker-compose.yml
    new file:   tests/test_app.py
  
```

Figure 3.1: Project files staged using Git

3.2 Commit Changes to Git

The newly created database model, Docker Compose configuration, and test file were committed to the Git repository using a descriptive commit message. The Git status confirms that the working tree is clean and the local main branch is one commit ahead of origin/main.

```
C:\Users\prana\smart-construction-project>git commit -m "Add database, testing and Docker Compose configuration"
[main d71e5a9] Add database, testing and Docker Compose configuration
3 files changed, 37 insertions(+)
create mode 100644 app/models.py
create mode 100644 docker-compose.yml
create mode 100644 tests/test_app.py

C:\Users\prana\smart-construction-project>git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use 'git push' to publish your local commits)

nothing to commit, working tree clean

C:\Users\prana\smart-construction-project>
```

Figure 3.2: Successful Git commit and clean working tree

3.3 Push Changes to Github.

The committed changes were pushed from the local main branch to the remote GitHub repository using git push origin main. The output confirms that the main branch was successfully updated on GitHub.

```
C:\Users\prana\smart-construction-project>git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 909 bytes | 909.00 KiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/pranathi-ravuru/smart-construction-project.git
d479281..d71e5a9  main -> main
```

Figure 3.3: Successful push of the project to GitHub

Section 3.4 – Verification of Project on GitHub.

The project was successfully pushed to the GitHub repository. The main branch contains the application source code, testing files, Docker configuration files, Docker Compose configuration, and requirements file. The latest commit, “**Add database, testing and Docker Compose configuration,**” is visible in the repository.

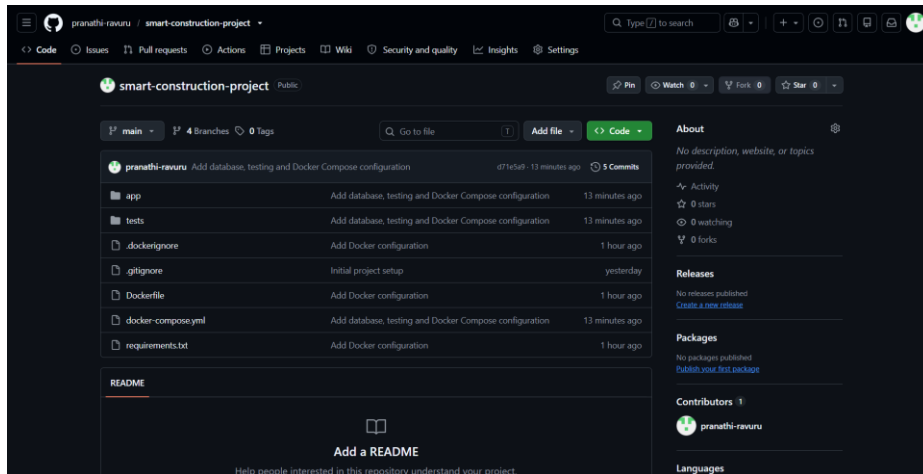


Figure 3.4: GitHub repository showing successfully pushed project files

4. Git Branching Strategy

4.1 Verify Docker Configuration

Docker and Docker Compose were verified successfully on the development system. Docker is used for containerizing the Smart Construction Project Management application, while Docker Compose is used to define and manage the application services.

```
C:\Users\prana\smart-construction-project>docker --version
Docker version 29.7.2, build a7dcaa6

C:\Users\prana\smart-construction-project>docker compose version
Docker Compose version v5.3.1
```

Figure 4.1: Docker and Docker Compose version verification

4.2 Verify Docker Files

```
C:\Users\prana\smart-construction-project>dir Dockerfile
Volume in drive C is Windows
Volume Serial Number is 2876-3C4F

Directory of C:\Users\prana\smart-construction-project

16-08-2026  21:42                182 Dockerfile
             1 File(s)              182 bytes
             0 Dir(s)  853,219,766,272 bytes free

C:\Users\prana\smart-construction-project>dir docker-compose.yml
Volume in drive C is Windows
Volume Serial Number is 2876-3C4F

Directory of C:\Users\prana\smart-construction-project

16-08-2026  22:18                604 docker-compose.yml
             1 File(s)              604 bytes
             0 Dir(s)  853,219,078,144 bytes free

C:\Users\prana\smart-construction-project>dir .dockerignore
Volume in drive C is Windows
Volume Serial Number is 2876-3C4F

Directory of C:\Users\prana\smart-construction-project

16-08-2026  21:42                49 .dockerignore
             1 File(s)               49 bytes
             0 Dir(s)  853,219,078,144 bytes free
```

Figure 4.2: Docker configuration files

4.3 Build the Docker Image

The Docker image for the Smart Construction Project Management System was successfully built using the Docker file. The build process completed all required stages, including loading the Python base image, copying the requirements file, installing dependencies, copying the application files, and exporting the final image.

```
C:\Users\prana\smart-construction-project>docker build -t smart-construction .
[*] Building 3.9s (11/11) FINISHED
=> [internal] load build definition from Dockerfile                                docker:desktop-linux 0.1s
=> => transferring dockerfile: 221B                                              0.1s
=> [internal] load metadata for docker.io/library/python:3.13-slim              2.6s
=> [auth] library/python:pull token for registry-1.docker.io                   0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 89B                                                  0.0s
=> [1/5] FROM docker.io/library/python:3.13-slim@sha256:ffb752e139c0a19692a43af8d8523b274222dd68eebad5d583b45c2201c6e30a
=> => resolve docker.io/library/python:3.13-slim@sha256:ffb752e139c0a19692a43af8d8523b274222dd68eebad5d583b45c2201c6e30a
=> [internal] load build context                                                0.1s
=> => transferring context: 723B                                                0.0s
=> CACHED [2/5] WORKDIR /app                                                    0.0s
=> CACHED [3/5] COPY requirements.txt .                                          0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt              0.0s
=> [5/5] COPY app ./app                                                         0.1s
=> exporting to image                                                          0.7s
=> => exporting layers                                                         0.2s
=> => exporting manifest sha256:ca326d071e6ed3ca5ea74ed65b9569800d96e04c9e7bcd08838daffdd38273b
=> => exporting config sha256:fb61ed4d3a2c926ad1dc45a85a912047b1163e92852f1b89ebb05a915fb37cc7
=> => exporting attestation manifest sha256:10ab0919286d7aac50e52254445dc357cbf1b3cdad723c1463bc285de8475fe
=> => exporting manifest list sha256:77782a7c0fd70f97ac7eada005ff358616a67046e7d165ef17f69fab40c5264a
=> => naming to docker.io/library/smart-construction:latest                    0.0s
=> => unpacking to docker.io/library/smart-construction:latest                 0.1s
```

Figure 4.3: Successful Docker image build

4.4 Docker Image Verification

The Docker image was successfully created and verified using the docker images command. The image smart-construction:latest is available locally with its corresponding image ID and size.

```
C:\Users\prana\smart-construction-project>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
smart-construction:latest	77782a7c0fd7	207MB	51.2MB	

Figure 4.4: Docker image successfully created

4.5 Run Docker container

The Smart Construction Project Management application was successfully started as a Docker container. Port 5000 of the host system was mapped to port 5000 of the container, enabling access to the Flask application through the local browser.

```
C:\Users\prana\smart-construction-project>docker run -d -p 5000:5000 smart-construction
d75f93ffdf8f4913ed2106fc0b8276ad7d044b9d29a0b25b36ce26de971ddb2f
```

```
C:\Users\prana\smart-construction-project>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
d75f93ffdf8f	smart-construction	"python -m app"	8 seconds ago	Up 8 seconds	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	amazing_lamarr

Figure 4.5: Smart Construction application running inside a Docker container

4.6 Browser/Application Verification.

The containerized Flask application was successfully accessed through the web browser using <http://localhost:5000>. The dashboard displays project information, project status, progress, task management details, and workforce information, confirming that the application is running successfully inside the Docker container.

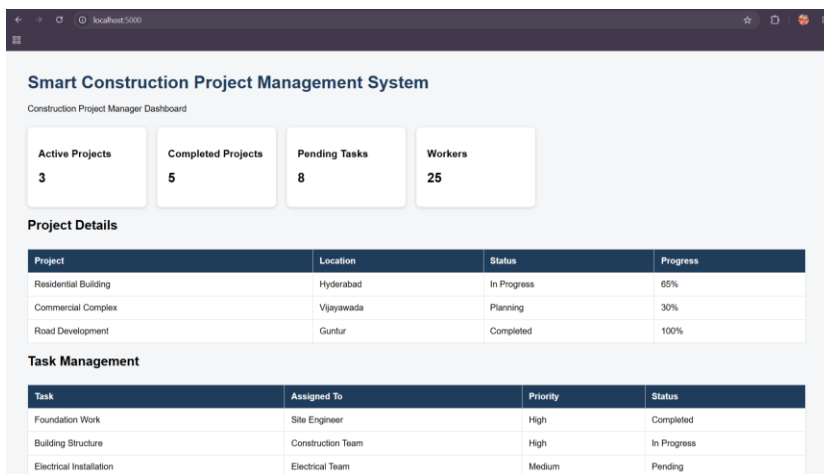


Figure 4.6: Smart Construction Project Management System running through Docker

5. Docker Compose

5.1 Stop the currently running container

The currently running Docker container was stopped using the docker stop command. The docker ps command was then executed to verify that the container was no longer running

```
C:\Users\prana\smart-construction-project>docker stop d75f93ffdf8f
d75f93ffdf8f
C:\Users\prana\smart-construction-project>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
C:\Users\prana\smart-construction-project>
```

Figure 5.1: Stopping the running Docker container and verifying its status

5.2 Docker Compose Configuration Validation

The Docker Compose configuration was verified using the docker compose config command. The output confirms that the database and web services, environment variables, ports, dependency configuration, volume, and network settings are correctly recognized by Docker Compose.

```
C:\Users\prana\smart-construction-project>docker compose config
name: smart-construction-project
services:
  db:
    container_name: smart-construction-db
    environment:
      POSTGRES_DB: construction_db
      POSTGRES_PASSWORD: postgres
      POSTGRES_USER: postgres
    image: postgres:16
    networks:
      default: null
    ports:
      - mode: ingress
        target: 5432
        published: "5432"
        protocol: tcp
    volumes:
      - type: volume
        source: postgres_data
        target: /var/lib/postgresql/data
        volume: {}
  web:
    build:
      context: C:\Users\prana\smart-construction-project
      dockerfile: Dockerfile
    container_name: smart-construction-web
    depends_on:
      db:
        condition: service_started
        required: true
    environment:
      DB_HOST: db
      DB_NAME: construction_db
      DB_PASSWORD: postgres
      DB_PORT: "5432"
      DB_USER: postgres
    networks:
      default: null
    ports:
      - mode: ingress
```

Figure 5.2 – Verification of Docker Compose Configuration

5.3 Start the complete application

The application was deployed using Docker Compose. The PostgreSQL database and Smart Construction web application containers were successfully created and started. The docker compose ps command was used to verify the running services and their port mappings.

```
#7 [1/5] FROM docker.io/library/python:3.11-slimsha256:ffb752e139c8a19692a43af8d8523b274222d66eeba5d583b45c2281cde3ba
#7 resolve docker.io/library/python:3.11-slimsha256:ffb752e139c8a19692a43af8d8523b274222d66eeba5d583b45c2281cde3ba
#7 resolve docker.io/library/python:3.11-slimsha256:ffb752e139c8a19692a43af8d8523b274222d66eeba5d583b45c2281cde3ba 0.1s done
#7 DONE 0.1s
#8 [2/5] WORKDIR /app
#8 CACHED
#9 [3/5] COPY requirements.txt .
#9 CACHED
#10 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#10 CACHED
#11 [5/5] COPY app . /app
#11 CACHED
#12 exporting to image
#12 exporting layers done
#12 exporting manifest sha256:aa224c97ec25624088856a3839ee8e2599a1d0b553d24853b14c211c123b88 0.8s done
#12 exporting config sha256:c9236dc97adcd4ed6693369378432999312983b2772487be38ac7c3956 0.8s done
#12 exporting attestation manifest sha256:07a0728b23ab977fde61786922786fc187c108cd4e850b31fa6a118a7b
#12 exporting attestation manifest sha256:07a0728b23ab977fde61786922786fc187c108cd4e850b31fa6a118a7b 0.8s done
#12 exporting manifest list sha256:99f380b0fdd3a13d1cd97a5c939c977fcd0e7c181d6e6cfaeb8b50b66 0.8s done
#12 naming to docker.io/library/smart-construction-project-web:latest done
#12 ununpacking to docker.io/library/smart-construction-project-web:latest 0.8s done
#12 DONE 0.3s

[*] up 21/21p provenance for metadata file
✓image postgres:16 pulled 24.4s
✓image smart-construction-project-web built 1.6s
✓network smart-construction-project_default created 0.1s
✓volume smart-construction-project_postgres_data created 0.4s
✓container smart-construction-db started 1.8s
✓container smart-construction-web started 0.9s

C:\Users\prana\smart-construction-project>docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED          STATUS          PORTS
smart-construction-db               postgres:16                         "docker-entrypoint.s..."           db         20 seconds ago  Up 19 seconds  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
smart-construction-web              smart-construction-project-web       "python -m app"                       web        20 seconds ago  Up 19 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
```

Figure 5.3 – Starting and Verifying Docker Compose Services

5.4 Verify the Web Application

The Smart Construction Project Management System was accessed through the browser using <http://localhost:5000>. The dashboard was successfully displayed, confirming that the web application container is running and accessible through the mapped port.

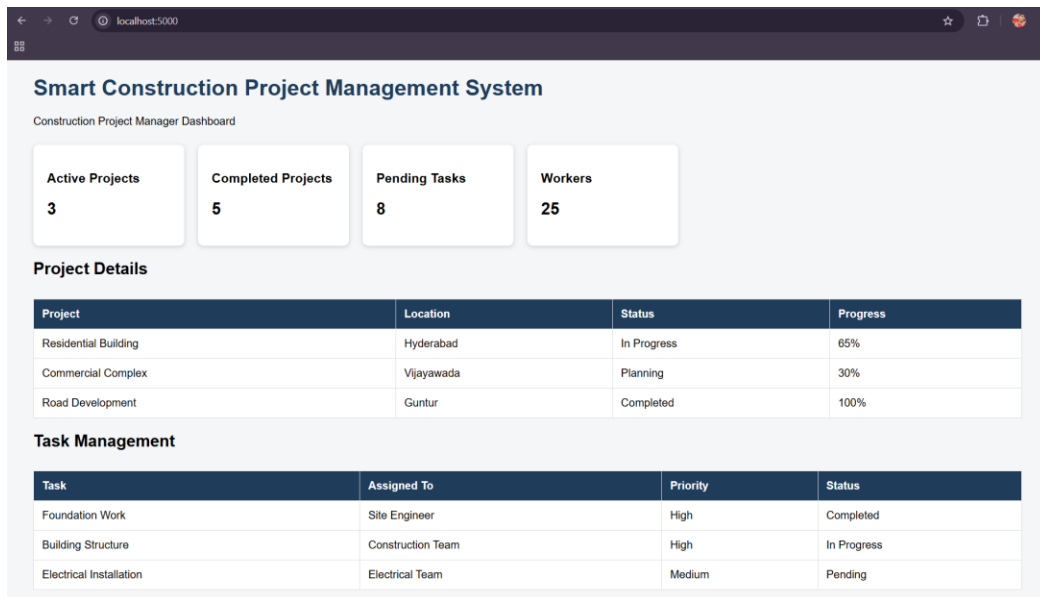


Figure 5.4 – Smart Construction Application Running Through Docker Compose

5.5 Verify Container Logs

```
C:\Users\prana\smart-construction-project>docker compose logs --tail=20
smart-construction-web | * Serving Flask app "app"
smart-construction-web | * Debug mode: off
smart-construction-web | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
smart-construction-web | * Running on all addresses (0.0.0.0)
smart-construction-web | * Running on http://127.0.0.1:5000
smart-construction-web | * Running on http://172.18.0.3:5000
smart-construction-web | Press CTRL+C to quit
smart-construction-web | 172.18.0.1 ~ [16/Aug/2026 18:27:57] "GET / HTTP/1.1" 200 -
smart-construction-db | waiting for server to shut down...2026-08-16 18:22:57.787 UTC [49] LOG: received fast shutdown request
smart-construction-db | 2026-08-16 18:22:57.796 UTC [49] LOG: aborting any active transactions
smart-construction-db | 2026-08-16 18:22:57.798 UTC [49] LOG: background worker "logical replication launcher" (PID 55) exited with exit code 1
smart-construction-db | 2026-08-16 18:22:57.800 UTC [50] LOG: shutting down
smart-construction-db | 2026-08-16 18:22:57.818 UTC [50] LOG: checkpoint starting: shutdown immediate
smart-construction-db | 2026-08-16 18:22:58.285 UTC [50] LOG: checkpoint complete: wrote 926 buffers (5.7%); 0 WAL file(s) added, 0 removed, 0 recycled;
smart-construction-db | write=0.077 s, sync=0.363 s, total=0.485 s; sync files=381, longest=0.006 s, average=0.002 s; distance=4273 kB, estimate=4273 kB; lsn=0/191F0F0, redo lsn=0/
191F0F0
smart-construction-db | 2026-08-16 18:22:58.291 UTC [49] LOG: database system is shut down
smart-construction-db | done
smart-construction-db | server stopped
smart-construction-db | PostgreSQL init process complete; ready for start up.
smart-construction-db | 2026-08-16 18:22:58.431 UTC [1] LOG: starting PostgreSQL 16.15 (Debian 16.15-1.pgdgl3+2) on x86_64-pc-linux-gnu, compiled by gcc
(Debian 14.2.0-19) 14.2.0, 64-bit
smart-construction-db | 2026-08-16 18:22:58.434 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
smart-construction-db | 2026-08-16 18:22:58.434 UTC [1] LOG: listening on IPv6 address "::", port 5432
smart-construction-db | 2026-08-16 18:22:58.451 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
smart-construction-db | 2026-08-16 18:22:58.467 UTC [65] LOG: database system was shut down at 2026-08-16 18:22:58 UTC
smart-construction-db | 2026-08-16 18:22:58.480 UTC [1] LOG: database system is ready to accept connections
smart-construction-db | 2026-08-16 18:27:58.535 UTC [63] LOG: checkpoint starting: time
smart-construction-db | 2026-08-16 18:28:02.854 UTC [63] LOG: checkpoint complete: wrote 45 buffers (0.3%); 0 WAL file(s) added, 0 removed, 0 recycled; w
rite=0.233 s, sync=0.019 s, total=0.320 s; sync files=12, longest=0.006 s, average=0.002 s; distance=261 kB, estimate=261 kB; lsn=0/1960560, redo lsn=0/1960
528
```

Figure 5.5 – Docker Compose Application Logs

5.6 Test the Database Connection

```
C:\Users\prana\smart-construction-project>docker compose exec db pg_isready -U postgres -d construction_db
/var/run/postgresql:5432 - accepting connections
```

Figure 5.6 – PostgreSQL Database Connection Verification

5.7 — Run Application Tests

The application tests were executed inside the Docker web container using pytest. One test item was successfully collected from tests/test_app.py, and the test passed successfully with a result of 1 passed

```
C:\Users\prana\smart-construction-project>docker compose exec web python -m pytest
===== test session starts =====
platform linux -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: /app
collected 1 item

tests/test_app.py . [100%]

===== 1 passed in 0.01s =====
```

Figure 5.7 – Successful Application Testing Inside Docker Container

5.8 — Final Service Verification

The final status of the Docker Compose services was verified using `docker compose ps`. Both the PostgreSQL database service and the Smart Construction web service were running successfully, confirming that the complete application environment was operational.

```
C:\Users\prana\smart-construction-project>docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
smart-construction-db	postgres:16	"docker-entrypoint.s..."	db	25 minutes ago	Up 25 minutes	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
smart-construction-web	smart-construction-project-web	"python -m app"	web	3 minutes ago	Up 3 minutes	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp

Figure 5.8 – Final Docker Compose Service Status

5.9 — Commit the final changes

```
C:\Users\prana\smart-construction-project>git add Dockerfile requirements.txt

C:\Users\prana\smart-construction-project>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   Dockerfile
        modified:   requirements.txt
```

Figure 5.9 – Final Docker Configuration Changes Staged for Commit

6. Push to GitHub

```
C:\Users\prana\smart-construction-project>git commit -m "Update Docker configuration for testing"
[main 27b0666] Update Docker configuration for testing
 2 files changed, 3 insertions(+), 1 deletion(-)

C:\Users\prana\smart-construction-project>git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 408 bytes | 408.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/pranathi-ravuru/smart-construction-project.git
 d71e5a9..27b0666  main -> main
```

Figure 6– Final Changes Pushed to GitHub

7. Final status check

```
C:\Users\prana\smart-construction-project>git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Figure 7 – Final Git Repository Status

8. Final Git Commit History and Final Changes Synchronized with GitHub

```
C:\Users\prana\smart-construction-project>git log --oneline -3
27b0666 (HEAD -> main, origin/main) Update Docker configuration for testing
d71e5a9 Add database, testing and Docker Compose configuration
d479281 (origin/feature/task-management, feature/task-management) Add Docker configuration

C:\Users\prana\smart-construction-project>git push origin main
Everything up-to-date
```

9. Github

The screenshot shows the GitHub interface for the repository 'smart-construction-project'. The repository is public and has 4 branches and 0 tags. The commit history is displayed, showing the latest commit '27b0666' by 'pranathi-ravuru' with the message 'Update Docker configuration for testing'. The file list shows the following files and their commit messages:

File	Commit Message	Time
app	Add database, testing and Docker Compose configuration	1 hour ago
tests	Add database, testing and Docker Compose configuration	1 hour ago
.dockerignore	Add Docker configuration	3 hours ago
.gitignore	Initial project setup	yesterday
Dockerfile	Update Docker configuration for testing	10 minutes ago
docker-compose.yml	Add database, testing and Docker Compose configuration	1 hour ago
requirements.txt	Update Docker configuration for testing	10 minutes ago

The README section is visible at the bottom, with a prompt to 'Add a README'.

10. Testing Evidence

The following screenshots should be included in the report:

- Git repository on GitHub.
- git init command.
- git status.
- Feature branch creation.
- Git commit.
- Git log.
- GitHub commit history.
- Dockerfile.
- Docker Compose file.
- docker compose build.
- docker compose up.
- docker ps.
- Application running at localhost:5000.
- PostgreSQL container running.
- Successful application operation.

11. Benefits of Git and Docker

11.1 Benefits of Git

Collaboration

Multiple developers can work on different features using separate branches.

Version Management

Git maintains the history of project changes.

Backup

The remote GitHub repository provides a backup of the source code.

Branching

Features can be developed independently without affecting the stable application.

Error Recovery

Previous versions can be restored using Git history.

Code Review

Pull requests allow developers to review changes before merging them.

11.2 Benefits of Docker

Portability

The application can run on different systems with Docker installed.

Consistency

The same application environment is used during development and deployment.

Isolation

The application and database operate in separate containers.

Scalability

Multiple application containers can be created when required.

Easy Deployment

The entire system can be started using docker compose up.

Maintainability

Dependencies and configuration are defined in project files rather than manually installed on every machine.

12. Challenges and Improvements

12.1 Challenges

Git Merge Conflicts

Conflicts may occur when multiple developers modify the same files.

Dependency Management

Different Python versions or library versions can cause application compatibility issues.

Docker Configuration

Incorrect ports, environment variables, or service names can prevent containers from communicating.

Database Persistence

Removing containers without a volume can result in loss of database data.

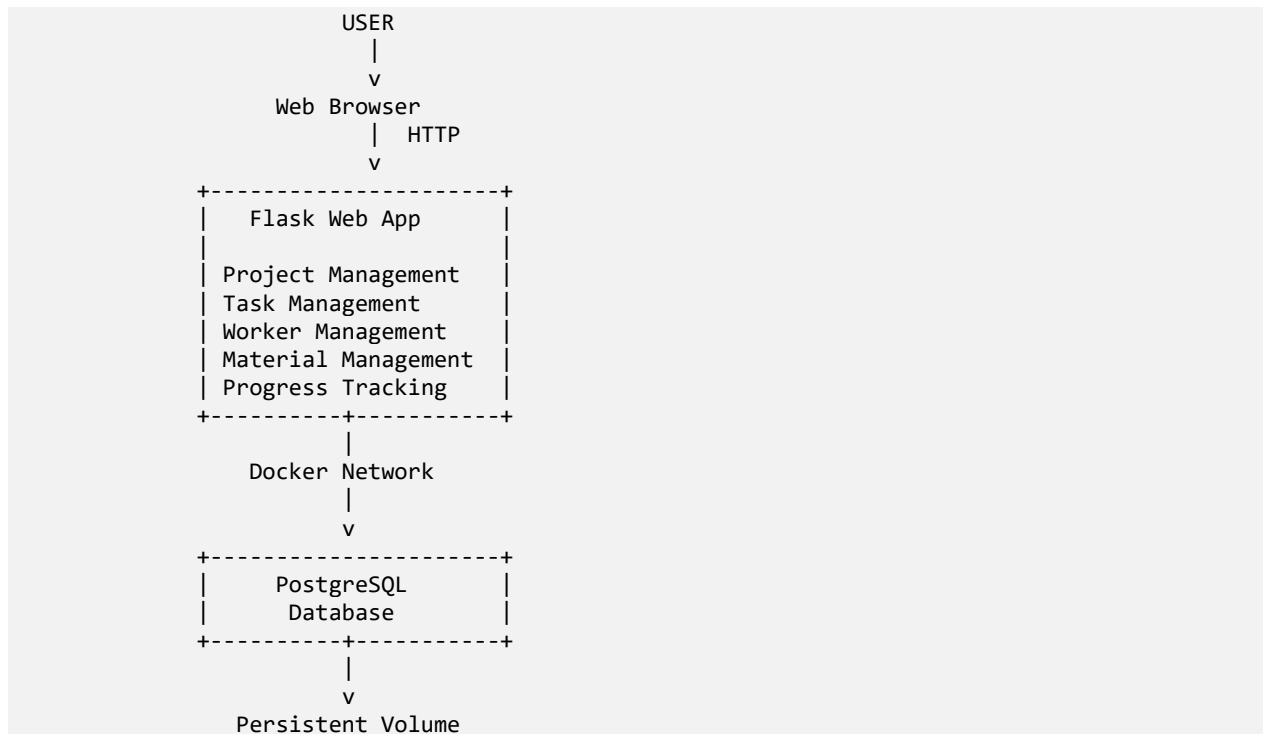
Container Debugging

Application errors must sometimes be investigated through container logs.

12.2 Improvements

- Add user authentication and authorization.
- Add an administrator dashboard.
- Add worker attendance management.
- Add material inventory tracking.
- Add project cost estimation.
- Add real-time project notifications.
- Add cloud deployment using AWS or Azure.
- Add CI/CD using GitHub Actions.
- Add automated Docker image builds.
- Add monitoring using Prometheus and Grafana.
- Use HTTPS for secure communication.
- Add automated database backups.

Overall System Architecture



Git + Docker Workflow

Developer -> Git Clone -> Feature Branch -> Develop Application
-> Commit Changes -> Push to GitHub -> Code Review / Merge
-> Docker Build -> Docker Compose
 -> Flask Container + PostgreSQL
-> Smart Construction Management System -> Testing -> Deployment

Conclusion

The Smart Construction Project Management System demonstrates the practical integration of Git version control and Docker containerization. Git provides systematic source-code management through commits, branches, merging, and remote repository synchronization. Docker packages the application and its dependencies into isolated and portable containers.

The combination of Git and Docker improves collaboration, reliability, deployment consistency, portability, scalability, and maintainability. The proposed system can be further extended with authentication, automated testing, CI/CD, cloud deployment, monitoring, and real-time project tracking.